Fundamentals of Programming & Computer Science
CS 15-112

Loops

Dr. Hend Gedawy

جامعة كارنيجي ميلون في قطر
Carnegie Mellon University Qatar

Outline

- Loops
  - For Loops
    - With Different forms of range
  - Nested For loops
  - While Loops
  - Loop control statements - Break & Continue

Announcements

- All CA OHs are added
- My OHs by appointment
- Pre-readings
  - By deadline, you get full credit
  - Post-deadline, you get partial credit
- Homework Interviews during mentor meetings
  - Starts On Wednesday for HW1
- HW2 is out
- Quiz 1 (About Week 1 Material) on Thursday

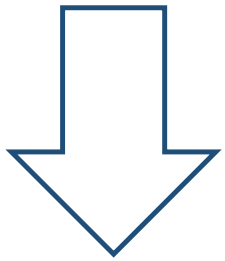
Why Do we need Loops?

```
print("1 squared = ", (1*1))
print("2 squared = ", (2*2))
print("3 squared = ", (3*3))
print("4 squared = ", (4*4))
print("5 squared = ", (5*5))
.
.
.
print("100 squared = ", (100*100))
```

```
>>> %Run Lect1.py
 1  squared is: 1
 2  squared is: 4
 3  squared is: 9
 4  squared is: 16
 5  squared is: 25
 6  squared is: 36
 7  squared is: 49
 8  squared is: 64
 9  squared is: 81
 10  squared is: 100
```

We are **repeatedly** executing a block of code

Print(X, "squared =" , (X * X))



With Loops we can do the same
in **2 Lines** of Code

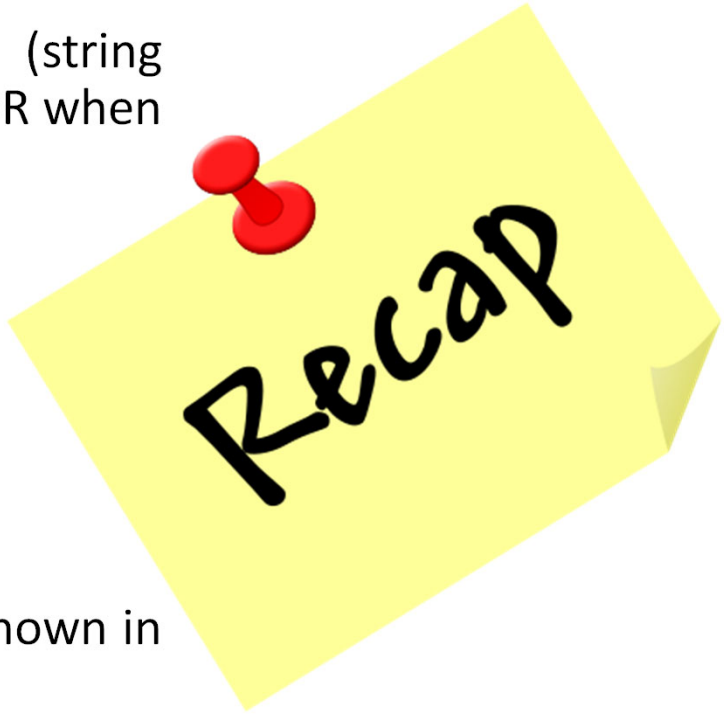
Infinite While Loop With Break-Example

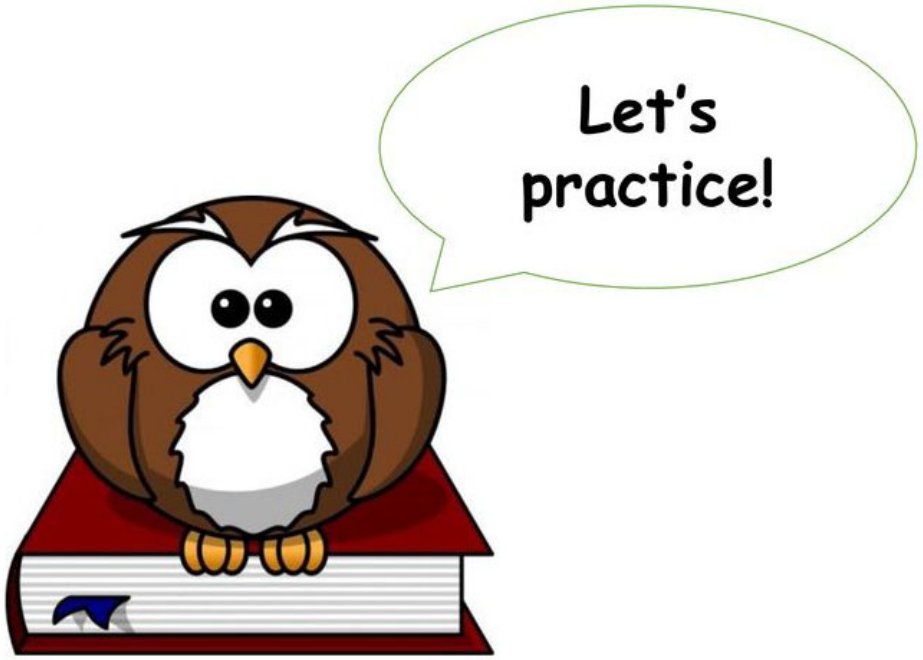
```
def readUntilDone():
    linesEntered = 0
    while (True):
        response = input("Enter a string (or 'done' to quit): ")
        if (response == "done"):
            break
        print("  You entered: ", response)
        linesEntered += 1
    print("Bye!")
    return linesEntered

linesEntered = readUntilDone()
print("You entered", linesEntered, "lines (not counting 'done').")
```

Last Lecture

Loops

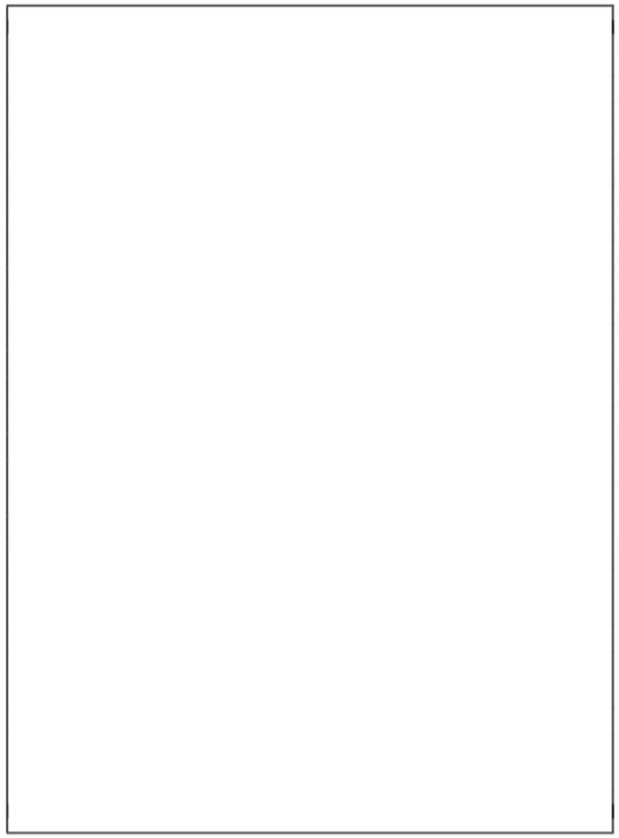
- For Loops
  - Better used when iterating over a sequence (string characters, list of items, a sequence of numbers) OR when number of iterations is known
  - With Different forms of range
    - range(n) – **[0,1,2…, n)**
    - range(s, n) – **[s, s+1, s+2, … , n)**
    - range(s, n, step)- **[s, s+step, s+2*step, … , n)**
- Nested For loops
- While Loops
  - Better used when the number of iterations is unknown in advance and is tied to a condition
- Loop control statements –
  - Break & Continue




Let's
practice!

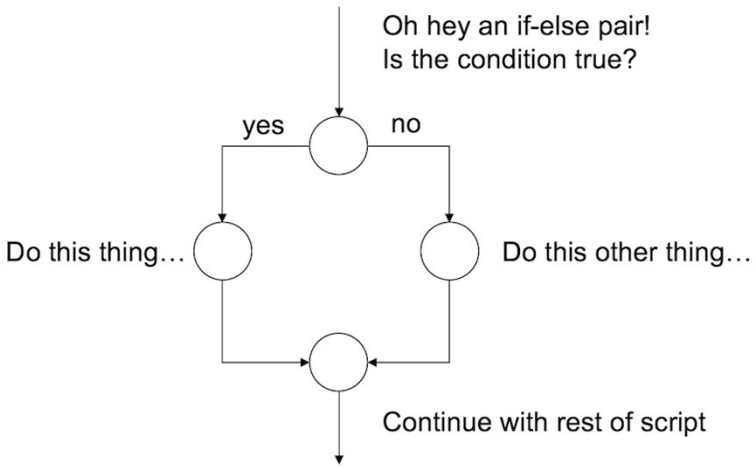
(b) (5 points) Indicate what the following program prints. Place your answers (and nothing else) in the box next to the code.

```
def ct1(m):
    x = 1
    while x < 6:
        if x%5 == 0:
            break
        x += 2
        print("x =", x)
    for y in range(m, m+3):
        if y % 3 == 0:
            print("mul3 = ", y)
        elif y % 2 == 0:
            print("even")
        else:
            x += y
    return x

print(ct1(4))
```

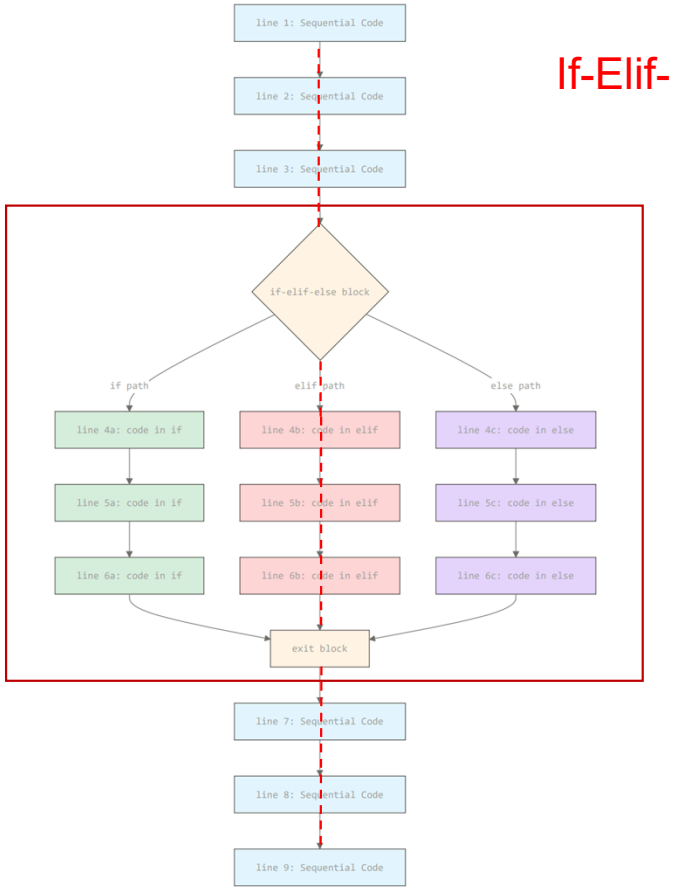


If-Elif-Else Block

If-Else



If-Elif-Else



Loops Practice –
Free Response Problem

Write a Function **isPrime(n)** which returns True if n is a prime number, and returns false otherwise

Call isPrime() on all values from 0 to 100

Loops Practice –
Free Response Problem

Write a Function **nthPrime(n)** which returns nth prime number

Call nthPrime(n) on all values from 1 to 10

Loops Practice –
Free Response Problem

Write a Function **CountPrimes(n)** which returns the count of prime number that are less or equal to n

Call **CountPrimes**(n) on all values from 1 to 10

3. (8 points) **Free Response:**

Write the function `longest42Run(n)`, that takes a positive integer `n` and returns the length of the longest run of the number of **42** within `n`'s digits.

Note: the definition of *run* is the same as the one given in the homework problem `longestDigitRun`, but this time applied to the two-digit sequence **42**.

For example...

- `longest42run(424200)` returns 2 because there are two **42** in sequence: $\underbrace{4242}_{2}00$
- `longest42run(42)` returns 1 because there is only one **42**: $\underbrace{42}_{1}$
- `longest42run(421142)` also returns 1 because not two consecutive **42**s appear in sequence: : $\underbrace{42}_{1}11\underbrace{42}_{1}$
- `longest42run(15112)` returns 0 because there is no **42** within the digits.
- `longest42run(142424242424200)` returns 6: $1\underbrace{424242424242}_{6}00$
- `longest42run(142424242004242)` returns 4: there are two runs of **42** of length 4 and 2, so you must return the longest: $1\underbrace{42424242}_{4}00\underbrace{4242}_{2}$

Loops Practice – CT

```
def f(x):
    print(x)
    return 42

def ct(x):
    counter = 0
    target = x
    for i in range(5, -1, -2):
        if counter == target:
            print("meet", counter)
            target += 1
        else:
            print("miss", target)
        counter += 2
    return f(counter)

ct(2)
```